

AK VisionFlow — Full Project Final Audit Report

Universal Architecture & Refactoring Program · Phases 0.1 – 0.8 · Prepared for client delivery/handover ·
September 18, 2026

Contents:

1. Scope and Method
2. Phase-by-Phase Summary (0.1 – 0.8)
3. Cross-Phase Architecture Verification Matrix
4. Consolidated Test Evidence
5. Security & Authorization Findings
6. Database & Migration Consistency
7. Accepted Conditions and Backlog Items
8. Production Readiness
9. Final Verdict

1. Scope and Method

This report covers the complete "Phase 0 — Universal Architecture & Refactoring" program: an eight-part effort to re-architect AK VisionFlow (the Khalid Eye Clinic system) from a single-industry optical/eyecare application into a universal, multi-tenant business platform capable of supporting additional industries later, without breaking anything the clinic already relies on today.

Every phase followed the same disciplined process: read the approved specification, inspect the existing code before touching it, implement, test, independently audit the result, fix anything found, retest, run a full regression of the entire application, and produce written evidence — then stop for explicit approval before starting the next phase. No phase proceeded without that approval. This report is a synthesis of all eight phases' own detailed, evidence-based reports (available individually in the project's `docs/` folder), re-verified together as a whole in Phase 0.8.

2. Phase-by-Phase Summary

0.1 — Existing System Audit & Feature Mapping **CLOSED**

A ground-truth audit of the system as it stood before any changes: a full database map, three API maps, a UI map, an Android app map, a test-coverage matrix, a feature-classification matrix, and a refactoring backlog — every claim checked directly against the code, nothing assumed. This audit is what every later phase was measured against, and it is what first discovered that the Owner Android app's backend had been quietly paused (see Phase 0.7).

0.2 — Universal Product Architecture CLOSED

Replaced the old assumption that every product is either an optical item or a medicine with a universal product model: a neutral core (name, price, stock, unit) plus optional industry-specific attribute tables for Optical and Medicine details, and a new product-variant capability. Every existing product was safely migrated and backfilled with zero data loss, and the optical and pharmacy screens continued to work exactly as before.

0.3 — Multi-Tenant / Company / Branch Architecture CLOSED

Introduced a "Company" layer between Tenant and Branch, so a business that operates more than one legal entity can be represented properly — while a business that only ever needed one branch sees no change at all. Twelve pre-existing gaps in branch-level data access were found and closed in the same pass (warehouses, stock transfers, procurement, journal entries, and several reports).

0.4 — Permissions, RBAC & Authorization Architecture CLOSED

Replaced scattered, hand-coded "which roles can do this" checks with one centralized permission system: a single catalog defining every action in the system and which staff roles may perform it, enforced consistently everywhere. Ultimately covered 25 resource types and 75 individual permissions across 239 role grants, migrated across roughly 40 backend modules without changing a single existing role's actual access — verified by direct comparison against the original behavior, not just by assumption. A safeguard was added so that no user, including an administrator, can ever grant themselves higher privileges.

0.5 — Core vs Industry Module Architecture CLOSED WITH CONDITIONS

Formally separated the parts of the system every business needs (products, customers, sales, purchases, accounting, etc.) from the parts specific to the optical/eyecare industry. Built a real, enforced "module registry" so an industry-specific feature can actually be switched on or off per business — this did not exist before; the settings previously only changed what fields appeared on a form, they didn't control real access. A tenant administrator can now genuinely enable or disable the Optical module for their business. Three minor, low-risk items were knowingly deferred rather than fixed immediately (see Section 7) and were accepted as conditions by the client.

0.6 — API & Backend Architecture CLOSED

A housekeeping pass across the whole backend: four separate pieces of logic that had been copy-pasted across dozens of files (page-number handling, date-range filters, invoice/order numbering, and duplicate-submission protection) were consolidated into shared, single-source utilities — reducing future bug risk with zero change in behavior, verified by full regression. One real security gap was found and fixed in the process: the basic owner dashboard was showing company-wide sales totals to branch-restricted staff instead of just their own branch's figures.

o.7 — Existing Owner Android Compatibility **CLOSED**

The Owner Android app (the mobile app previously built for the business owner to check sales, alerts, and AI-driven insights from their phone) had been paused mid-way through its own development, before this program began. This phase determined exactly what still worked, what needed adjustment, and what should stay deferred — then restored the app to a fully working, fully tested state *without switching it on for real use*. Two genuine defects left over from how the app had been paused were found and fixed: the backend database definition did not match what had actually been built, and a scheduled job that was supposed to send push notifications had been silently disconnected. Per the client's explicit instruction, this app remains off for real tenants and devices until a future phase is explicitly asked to turn it on.

o.8 — Architecture Testing & Approval **APPROVED WITH CONDITIONS**

The final gate: a complete, fresh, whole-program re-verification of everything above, run together rather than phase by phase — including, for the first time, running the entire backend test suite (all 415 tests) against a database built completely from scratch, to prove the full migration history applies cleanly with no leftover inconsistency. No new features were added and no changes were made "to improve the score" — this phase's job was strictly to verify, and it found no blocking issues. Full results are in Section 4.

3. Cross-Phase Architecture Verification Matrix

Area	Status	Basis
Universal Product Architecture	Verified	Automated test suite passing; industry-specific fields confirmed unaffected
Tenant → Company → Branch → Warehouse hierarchy	Verified	Dedicated isolation test suites passing
Centralized permissions (RBAC)	Verified	33 dedicated authorization tests passing; every frontend permission check cross-checked against the backend catalog with zero mismatches
Core vs Industry Module separation	Verified	Confirmed the universal core has zero dependency on optical-specific code, including a live test proving the core keeps working with the Optical module switched off entirely
Module registry and activation	Verified	24 registered modules (Core/Universal/Industry); real on/off enforcement confirmed by test
API & backend architecture	Verified	Duplication removed from 44 file-level instances across four categories; one authorization gap found and closed
Owner Android compatibility	Verified (restored, not activated)	Full backend and Android test suites passing; app remains switched off for real use
Cross-tenant / company / branch / warehouse isolation	Verified	Dedicated isolation tests across every module family
Optical/Medical functionality	Verified — zero regression	Full clinical workflow test suite (32 tests: patients, appointments, examinations, prescriptions, optical orders, labs) passing throughout every phase

Database/schema/migration consistency	Verified	Full migration history applies cleanly to a brand-new database with zero conflicts or drift
Frontend/backend authorization consistency	Verified	Every permission referenced in the user interface exists, unchanged, in the backend's authorization catalog
Test-environment safety	Verified	Every test throughout the program ran against disposable local test databases; production was never written to

4. Consolidated Test Evidence (Phase 0.8 final re-verification)

Suite	Result	Notes
Backend — full regression, including the Owner Android backend	415 / 415 tests passed (18 of 18 suites)	Run against a database built entirely from scratch for this check
Frontend — automated tests	76 / 76 tests passed (19 of 19 files)	
Frontend — code-quality check	Clean — no new issues	
Frontend — production build	Succeeds cleanly	
Owner Android app — automated unit tests	58 / 58 tests passed	Fresh, forced-clean build (not a cached/reused result)
Owner Android app — code-quality check (lint)	No issues found	
Owner Android app — build	Succeeds ; installable app file produced	
Owner Android app — on-device/emulator testing	Not performed	No physical Android device and no working emulator are available in this project's current environment. This is disclosed rather than assumed — see Section 8.

One transient, environment-level hiccup was investigated during this final round: an occasional "database connection" error appeared in a handful of test runs, always resolving cleanly on an immediate retry, never a genuine logic problem. This was traced conclusively to the local test-database software used purely for development testing occasionally taking a moment to accept a brand-new connection — a characteristic of the local testing tool, not of the application, and never observed against anything resembling production.

5. Security & Authorization Findings

- No cross-tenant data leakage found anywhere in the system — verified by dedicated adversarial tests attempting exactly that, across every module.
- No self-privilege-escalation path exists — a user, including an administrator, cannot grant themselves higher access.
- Two real authorization gaps were found and closed during this program (a set of branch-visibility gaps in Phase 0.1/0.3, and one dashboard-scoping gap in Phase 0.6) — both are now covered by automated tests that will fail if either regresses.
- The Owner Android app's security posture (encrypted on-device token storage, enforced HTTPS in release builds, no logged credentials, minimal device permissions) was independently reviewed and found sound; see the separate Owner Android App Final Audit Report for full detail.

6. Database & Migration Consistency

Every schema change made across all eight phases was verified to apply cleanly, in the correct order, to a database built completely from nothing — this is the strongest possible check that the migration history is internally consistent and that a fresh production environment (or a disaster-recovery restore) would come up correctly. One inconsistency was found and corrected during Phase 0.7: a database change belonging to the paused Owner Android work had, it turned out, already been quietly present in every test database used throughout this program, but the system's own internal definition of its data model had never been updated to match it. This has been corrected; it was never applied to the production database, and still has not been.

7. Accepted Conditions and Backlog Items

The following are **known, deliberately deferred items — not defects**. Each was explicitly reviewed and accepted by the client at the time its phase closed, and none is scheduled to be revisited unless a future phase specifically calls for it.

#	Item	Why deferred
1	The Optical retail workflow and the Eye-Clinic clinical workflow are switched on/off together as one module, not two independent ones	They are directly linked in the current data model (an optical order can reference a clinical prescription); splitting them would be a significant data-model change with no current business need
2	The AI insights engine and the WhatsApp/communication engine are not individually restricted by the new module on/off system	Both simply show nothing for a business with no optical activity — there is no data-leak risk, only an unenforced (and currently unnecessary) boundary
3	One inactive, unused ledger account entry related to Optical revenue remains in the universal accounting setup regardless of whether Optical is switched on	It is never actually created for a business that doesn't use it; removing the label itself would require a larger, currently unjustified change to the accounting engine
4	The Owner Android app remains switched off for real tenants and devices	Restored to a fully working, fully tested state, but deliberately not activated per the client's explicit instruction — activation is reserved for a future phase

8. Production Readiness

Item	Status
Universal Core, Company/Branch/Warehouse hierarchy, RBAC, Core/Industry separation	Verified and in active use today
Optical/Medical (clinic) functionality	Verified, fully regression-tested, unchanged in behavior
Owner Android app — code correctness	Verified locally; not deployed
Owner Android app — database change applied to production	Not yet done — requires client authorization
Owner Android app — production server address, real push notification service	Not yet configured
Owner Android app — on-device verification	Not yet possible in any environment available to this project

The web application (the system used day-to-day by clinic staff) is the subject of this entire program and is verified, tested, and already in its current working state — this report does not identify any reason it cannot continue to be used as

it is today. The remaining items above are specific to the separately-scoped Owner Android app, which is intentionally not part of daily operations yet.

9. Final Verdict

 APPROVED WITH CONDITIONS — READY FOR PHASE 1

Every architectural area reviewed across this eight-part program is verified, with fresh, complete, evidence-based testing behind it: 415 of 415 backend tests, 76 of 76 frontend tests, and 58 of 58 Owner Android tests, all passing, run against a database built from scratch to prove there is no hidden inconsistency. No blocking issues were found. The four items listed in Section 7 are accepted, previously-approved conditions carried forward as known context for future work — not open problems requiring resolution before proceeding. The system is ready for the next phase of development.